



Taller Práctico de Base de Datos: Sistema ASOGASINGA

Contexto del Negocio

La **Asociación de Ganaderos Sin Ganado (ASOGASINGA)** requiere centralizar y automatizar el control de sus operaciones ganaderas, las cuales incluyen la gestión de socios, fincas, inventario de ganado, control de producción de leche, esquemas de vacunación, personal y alimentación.

Para optimizar la lógica del negocio dentro del servidor y garantizar la seguridad de la información, se solicita la implementación del esquema relacional provisto y el desarrollo de componentes programables (Procedimientos Almacenados y Funciones), así como un esquema estricto de seguridad basado en roles.

Requerimientos del Proyecto

1. Implementación del Esquema (DDL y DML)

- Ejecutar el script base proporcionado para la creación de las 12 tablas (socios, municipios, fincas, tipos_ganado, ganado, veterinarios, vacunas, vacunacion, produccion_leche, empleados, alimentos, alimentacion).
- Asegurar la integridad referencial mediante las llaves primarias y foráneas definidas.

2. Procedimientos Almacenados (Stored Procedures)

Desarrolle **5 procedimientos almacenados** para automatizar tareas operativas de la asociación. Deben incluir parámetros de entrada y/o salida según corresponda:

- **sp_RegistrarProduccionLeche:** Recibe el código de arete del ganado, la fecha y la cantidad de litros. Debe buscar el ganado_id correspondiente y registrar la producción, validando previamente que el ganado sea de tipo "Lechero" y sexo "Hembra".
- **sp_ActualizarSalarioEmpleado:** Recibe el empleado_id y un porcentaje de aumento (ej. 10.00 para un 10%). Debe actualizar el salario en la tabla empleados.
- **sp-TrasladarGanadoFinca:** Recibe el ganado_id y el finca_id de destino. Modifica la ubicación actual del animal.
- **sp_RegistrarVacunacion:** Recibe el código de arete del animal, el nombre de la vacuna, el id del veterinario y las observaciones. Debe insertar el registro con la fecha actual del sistema.



- **sp_ReporteGastoAlimento:** Recibe el ganado_id y un rango de fechas. Debe devolver el total gastado en alimentación para ese animal en ese periodo (multiplicando cantidad_kg por el costo del alimento).

3. Funciones Creadas por el Usuario (User-Defined Functions - UDF)

Desarrolle **5 funciones** que faciliten la consulta de datos calculados dentro del sistema:

- **fn_CalcularEdadMeses:** Recibe el ganado_id y devuelve un entero con la edad actual del animal calculada en meses a partir de su fecha_nacimiento.
- **fn_TotalLitrosFinca:** Recibe el finca_id y un rango de fechas, devolviendo el total de litros de leche producidos por todos los animales de esa finca en ese periodo.
- **fn_PromedioPesoPorRaza:** Recibe el nombre de una raza (ej. 'Holstein') y devuelve el peso promedio de los animales pertenecientes a ella.
- **fn_ContarGanadoPorSocio:** Recibe el socio_id y devuelve la cantidad total de cabezas de ganado que posee a través de todas sus fincas.
- **fn_CostoTotalAlimentacion:** Recibe un ganado_id y devuelve la sumatoria histórica del costo de los alimentos consumidos por ese animal.

4. Seguridad: Usuarios, Roles y Permisos

Defina e implemente una estrategia de seguridad mediante la creación de **5 usuarios con diferentes roles** y privilegios limitados según el principio de menor privilegio:

1. **usuario_admin (Rol: Administrador del Sistema)**
 - *Permisos:* Control total sobre la base de datos (ALL PRIVILEGES). Puede crear, modificar y eliminar tablas, así como ejecutar cualquier procedimiento o función.
2. **usuario_veterinario (Rol: Personal de Salud Animal)**
 - *Permisos:* Permisos de lectura (SELECT) en las tablas ganado y fincas. Permisos de inserción (INSERT) en las tablas vacunacion y vacunas. Permiso de ejecución (EXECUTE) en el procedimiento sp_RegistrarVacunacion.
3. **usuario_operador (Rol: Encargado de Finca / Producción)**
 - *Permisos:* Permisos de lectura y escritura (SELECT, INSERT, UPDATE) en las tablas produccion_leche, alimentacion y ganado. Permiso de ejecución en sp_RegistrarProduccionLeche y fn_CalcularEdadMeses. No debe ver datos de la tabla empleados.
4. **usuario_rrhh (Rol: Recursos Humanos)**
 - *Permisos:* Control total (SELECT, INSERT, UPDATE, DELETE) exclusivamente sobre la tabla empleados. Permiso de ejecución para el procedimiento sp_ActualizarSalarioEmpleado.



5. **usuario_auditor (Rol: Auditor Externo / Consulta)**

- **Permisos:** Únicamente permisos de lectura (SELECT) en **todas** las tablas del sistema. Tiene estrictamente prohibido realizar operaciones DML (INSERT, UPDATE, DELETE) o alteraciones estructurales.

5. Automatización y Reglas de Negocio Avanzadas (Triggers y Eventos)

Para robustecer la integridad transaccional, trazabilidad y mantenimiento automático del sistema ASOGASINGA, implemente los siguientes 3 disparadores (*triggers*) y 2 eventos programados:

Trigger 1: **trg_ValidarPesoGanado (BEFORE INSERT / BEFORE UPDATE)**

- **Tabla:** ganado
- **Objetivo:** Garantizar la coherencia biológica en los registros de peso.
- **Regla de negocio:** Intercepta la inserción o actualización de un animal. Si el valor de peso_kg es menor o igual a 0 o superior a 1500.00 kg, debe abortar la transacción lanzando un error mediante SIGNAL SQLSTATE '45000' con el mensaje: 'Error: El peso ingresado está fuera de los rangos biológicos válidos (1 - 1500 kg)'.

Trigger 2: **trg_AuditarAumentoSalario (AFTER UPDATE)**

- **Tabla:** empleados
- **Objetivo:** Mantener trazabilidad sobre los cambios salariales realizados por RRHH o procedimientos almacenados.
- **Regla de negocio:** Se dispara únicamente cuando NEW.salario <> OLD.salario. Debe insertar automáticamente un registro en una tabla auditoria_salarios (que almacena empleado_id, salario_anterior, salario_nuevo, diferencia y fecha_modificacion), permitiendo auditar incrementos o deducciones.

Trigger 3: **trg_ValidarIntervaloVacunacion (BEFORE INSERT)**

- **Tabla:** vacunacion
- **Objetivo:** Evitar la sobredosis o revacunación accidental del ganado.
- **Regla de negocio:** Antes de registrar una dosis, verifica si el animal ya recibió esa misma vacuna en los últimos 30 días respecto a la fecha del registro. En caso afirmativo, cancela la inserción con SIGNAL SQLSTATE '45000' indicando: 'Error: El animal ya recibió esta vacuna en un periodo menor a 30 días'.



Evento 1: evt_DepuracionAuditoriaMensual (Recurrente Mensual)

- **Frecuencia:** EVERY 1 MONTH
- **Objetivo:** Mantenimiento de almacenamiento en tablas transaccionales de trazabilidad.
- **Acción:** Depura (elimina) todos los registros de la tabla auditoria_salarios (o de logs del sistema) que tengan más de 6 meses de antigüedad (`fecha_modificacion < NOW() - INTERVAL 6 MONTH`), registrando en una bitácora la cantidad de filas purgadas.

Evento 2: evt_CierreSemanalProduccionLeche (Recurrente Semanal)

- **Frecuencia:** EVERY 1 WEEK (programado todos los domingos a las 23:59:00).
- **Objetivo:** Consolidar métricas operativas de la asociación para reportes rápidos.
- **Acción:** Ejecuta una consulta que agrupa la producción total de leche de la semana por cada finca e inserta el consolidado en una tabla resumen_semanal_fincas (`finsa_id`, `litros_totales`, `semana_anio`, `fecha_cierre`), evitando cálculos costosos en reportes históricos.

Entregables

1. Script SQL con la estructura base e inserts provistos ejecutados con éxito.
2. Script SQL con la codificación y prueba de los 5 Procedimientos Almacenados.
3. Script SQL con la codificación y prueba de las 5 Funciones.
4. Script SQL con la creación de los usuarios, asignación de contraseñas y la respectiva asignación de privilegios (GRANT).